

Software Design Document Project Name:

Airport Runway Management Simulator Course:

Software Development 1

Programming Language:

Java

Prepared By:

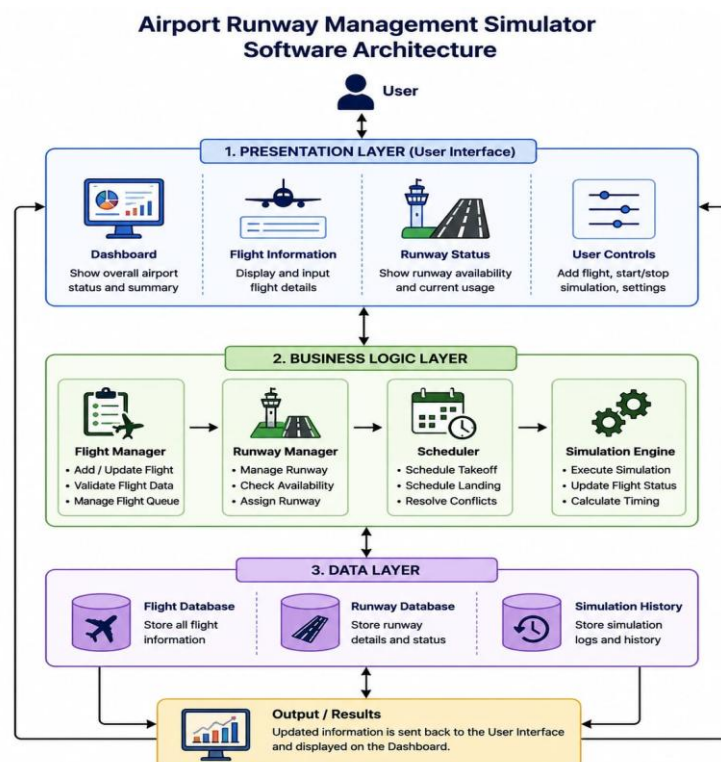
(REFFAT ARA ISLAM ANNI)

1.Introduction

What I Will Do

1.1 Design Software Architecture

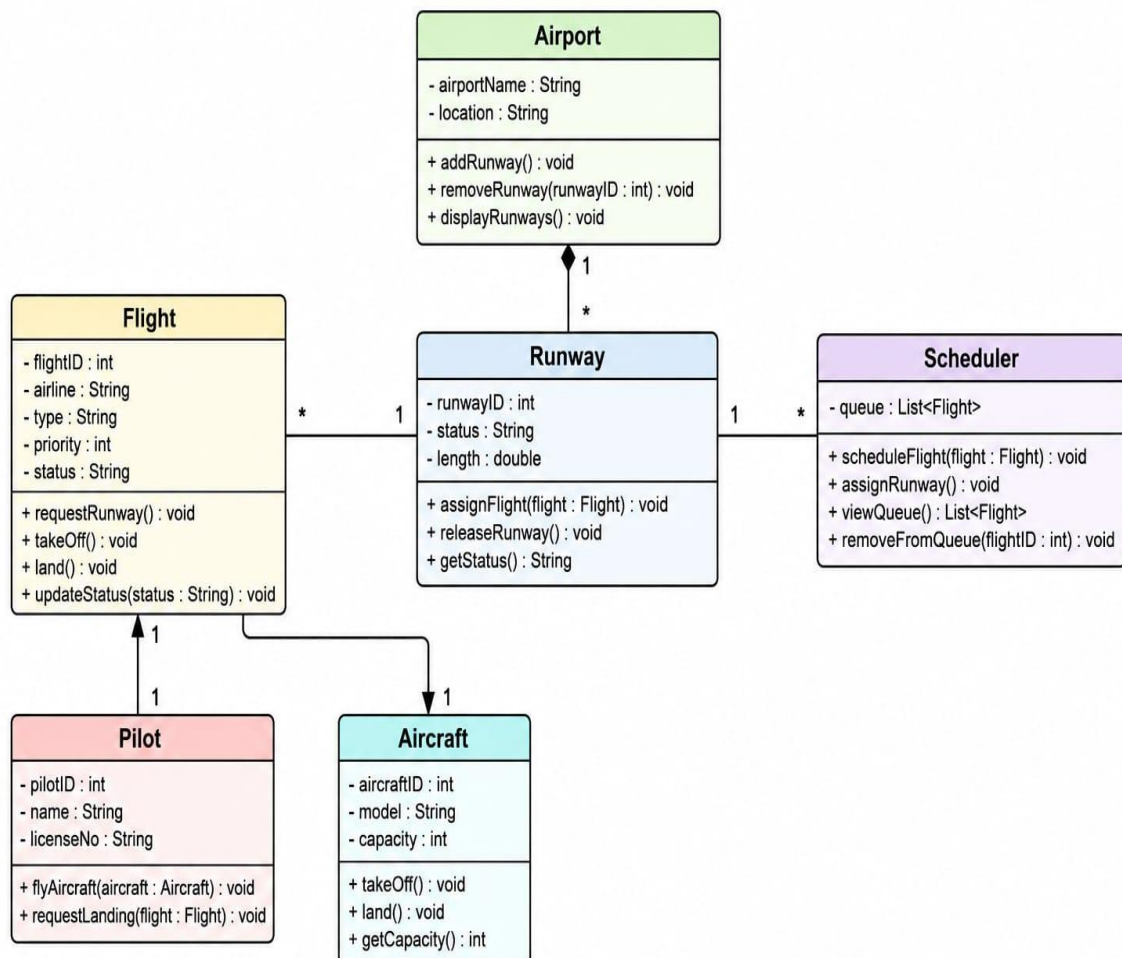
During the second week of the project, I will design the overall software architecture for the Airport Runway Management Simulator. The system will be divided into three major layers: Presentation Layer (User Interface), Business Logic Layer, and Database Layer. The architecture will also define modules such as Flight Management, Runway Management, User Management, Emergency Management, and Report Generation. This design will make the software organized, scalable, and easy to maintain.



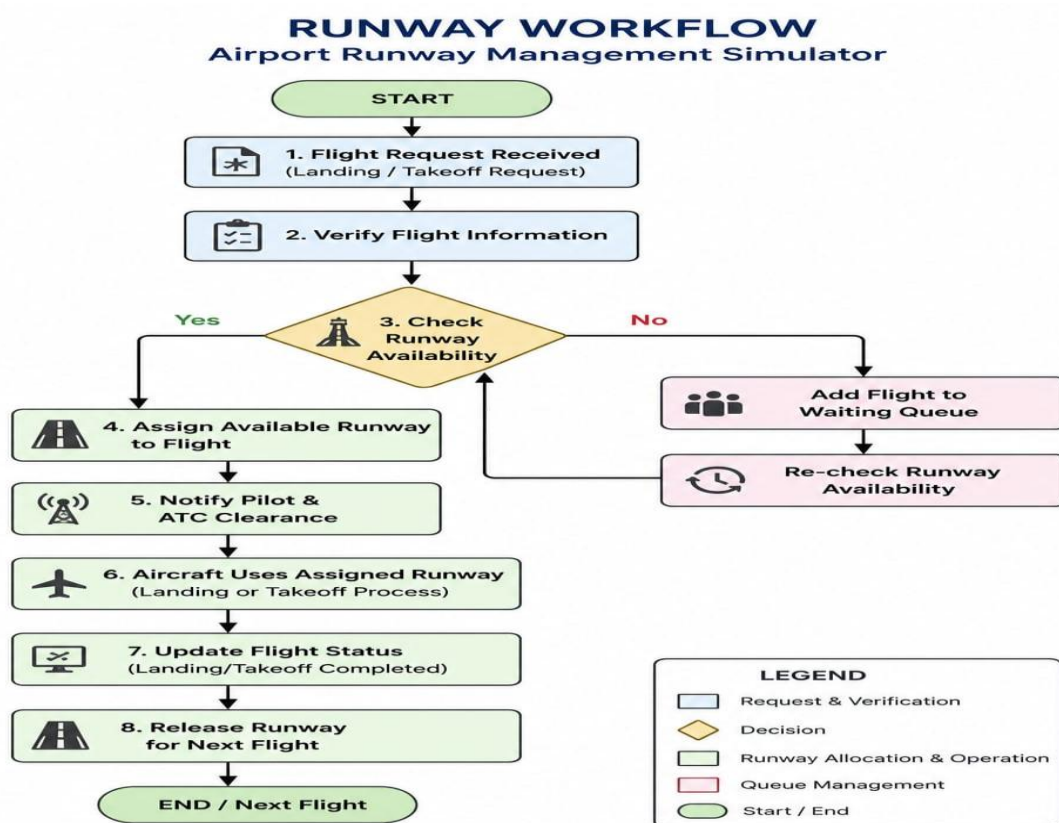
1.2 Create UML Class Diagram

I will prepare a UML Class Diagram to represent the structure of the software. The diagram will include important classes such as Flight, Runway, User, Airport Controller, and Report. Each class will contain its attributes, methods, and relationships. The UML diagram will serve as the blueprint for implementation and help reduce design errors.

UML Class Diagram
Airport Runway Management Simulator

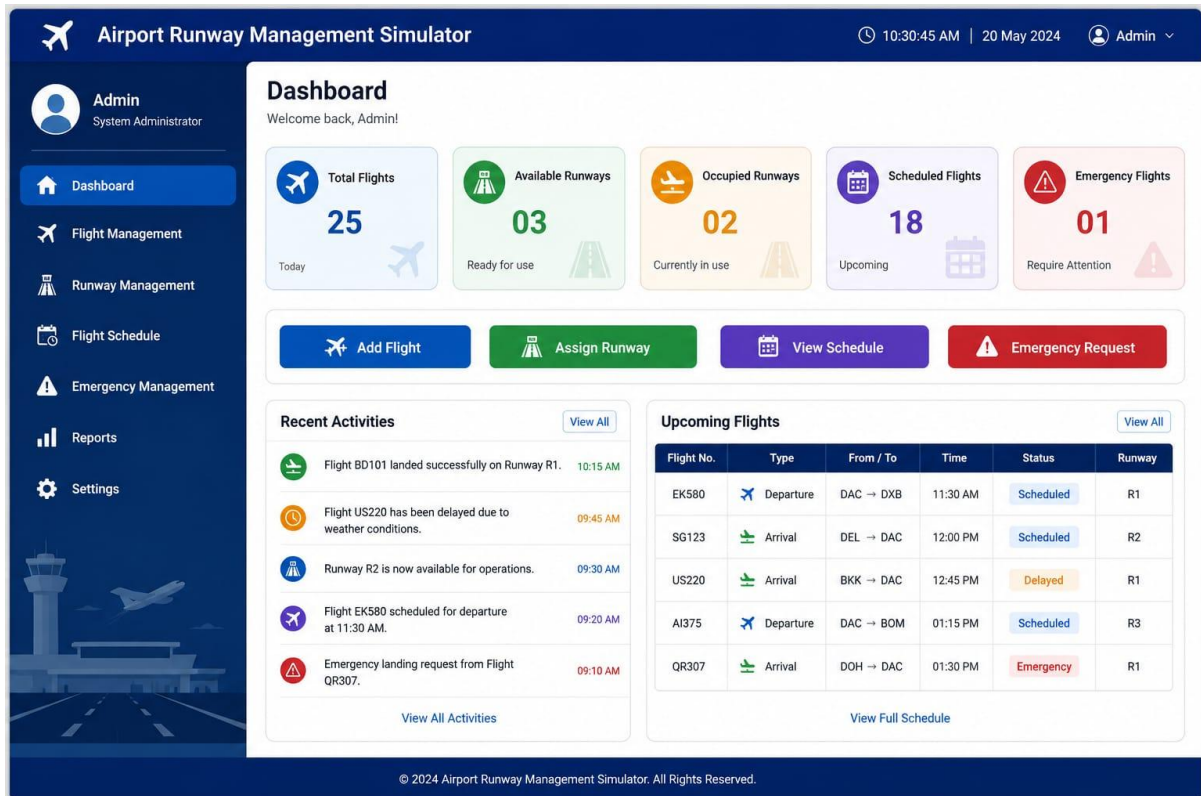


1.3 Design Runway Workflow I will design the complete runway workflow beginning from a flight request. The system will check runway availability, assign an available runway, update flight status, manage waiting queues when runways are occupied, handle emergency priority, and finally release the runway after landing or takeoff. This workflow will ensure efficient and safe runway operations.



1.4 Plan User Interface

I will design a simple and user-friendly interface for airport staff. The interface will include Login, Dashboard, Flight Management, Runway Management, Emergency Management, and Reports pages. The dashboard will display available runways, flight schedules, and alerts, enabling users to perform operations efficiently.



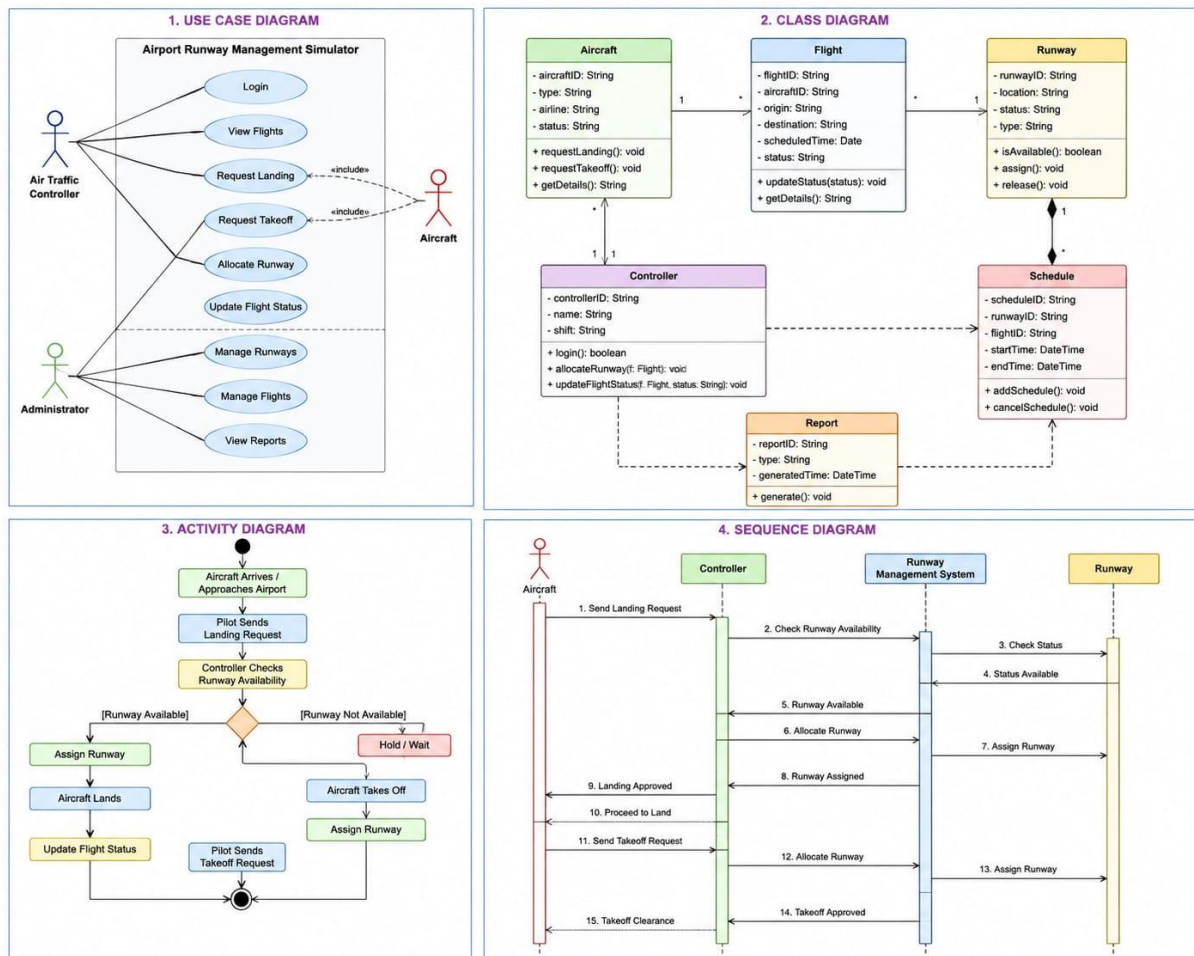
2. What I Need to Learn

2.1 OOP (Object-Oriented Programming)

Object-Oriented Programming (OOP) is a programming paradigm used to design software using classes and objects. For this project, I need to understand the core concepts of OOP, including Classes, Objects, Encapsulation, Inheritance, Polymorphism, and Abstraction. These concepts will help me write modular, reusable, and maintainable Java code. OOP will also make it easier to organize different modules such as Flight Management, Runway Management, and User Management.

2.2 UML (Unified Modeling Language)

UML is a standard visual language used for designing software systems before implementation. I need to learn how to create UML diagrams, especially the Class Diagram, to represent classes, attributes, methods, and relationships. Using UML will help in planning the project structure, improving communication among team members, and reducing design errors.



2.3 Java Classes & Objects

Since the project will be developed using Java, I need to learn how to create classes, define attributes and methods, use constructors, and create objects. I also need to understand how objects communicate with each other through methods and how Java classes can represent real world entities such as Flight, Runway, User, and Airport Controller. This knowledge will help me implement the software design efficiently.

3.

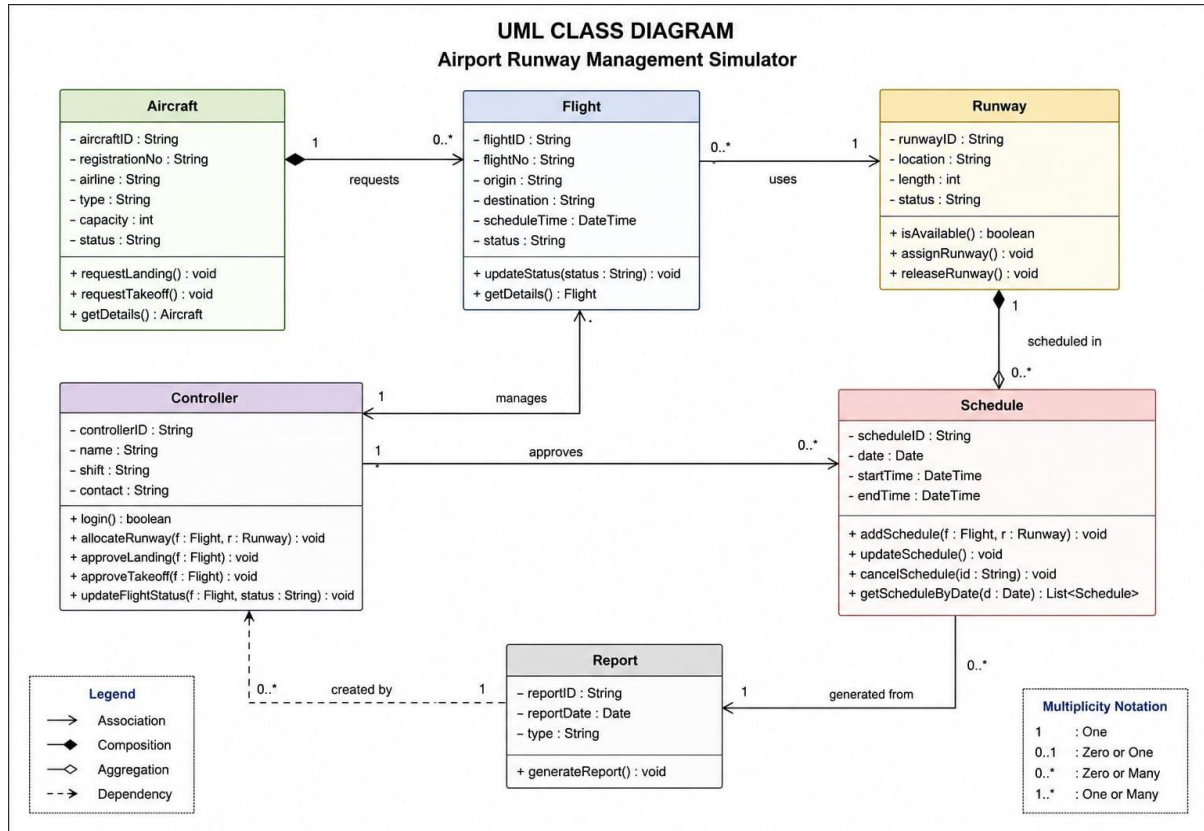
GitHub Upload

3.1 System Design Document

The System Design Document contains the complete software architecture, module descriptions, database overview, workflow explanation, and technology stack used in the Airport Runway Management Simulator. It serves as the primary technical document for understanding how the system is designed before implementation. This document helps developers maintain consistency during coding and future development.

3.2 Class Diagram

The Class Diagram illustrates the structure of the software using UML. It includes major classes such as Flight, Runway, User, Airport Controller, and Report. Each class contains attributes, methods, and relationships such as association and dependency. Uploading the Class Diagram to GitHub allows all team members to access and understand the software design easily.



3.3 Project Structure

The Project Structure organizes all project files into appropriate folders. Typical folders include src/model for model classes, src/controller for business logic, src/view for user interface, docs for documentation, and resources for configuration files. Maintaining a clean project structure in GitHub improves collaboration, readability, and maintainability.

4.

What I Will Demonstrate

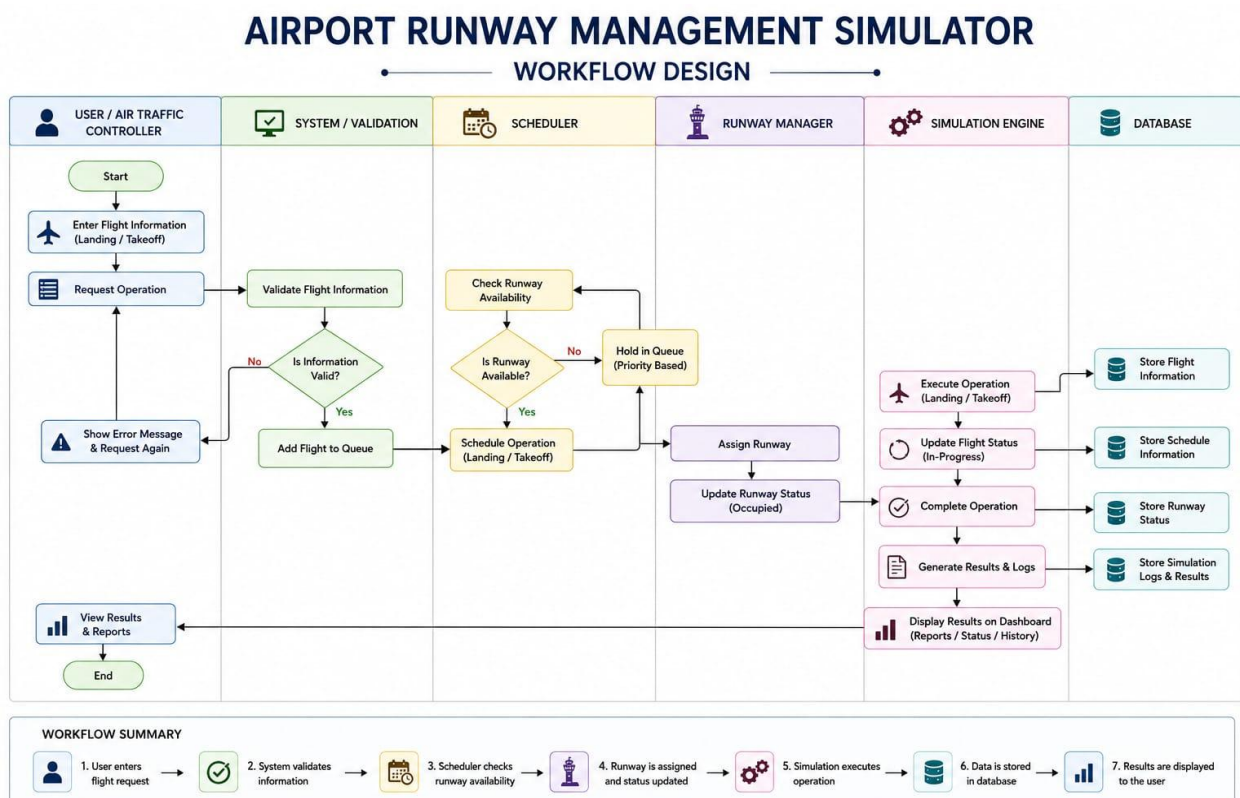
4.1 Software Architecture

During the Week 2 presentation, I will demonstrate the software architecture of the Airport Runway Management Simulator. The system follows a layered architecture consisting of the Presentation Layer, Business Logic Layer, and Database Layer. I will explain how users interact with the application through the user interface, how business logic processes flight and runway operations, and how data is stored and retrieved from the database. I will also describe the responsibilities of important modules such as:

Flight Management, Runway Management, User Management, Emergency Management, and Report Generation. This demonstration will show that the software is well organized, scalable, and easy to maintain.

4.2 Workflow Design

I will demonstrate the complete runway workflow used in the simulator. The workflow starts when a flight requests permission for landing or takeoff. The system checks runway availability, assigns an available runway, updates the flight status, and releases the runway after the operation is completed. If all runways are occupied, the flight is placed in a waiting queue until a runway becomes available. Emergency flights receive the highest priority and are processed before normal flights. This workflow demonstrates how the simulator manages airport runway operations safely, efficiently, and systematically.



Conclusion

The second week of the **Airport Runway Management Simulator** project focused on developing the software design. During this phase, the software architecture was planned, the UML Class Diagram was prepared, the runway workflow was designed, and the user interface was planned. In addition, important concepts such as Object-Oriented Programming (OOP), UML, and Java Classes & Objects were studied to build a strong foundation for the implementation phase.